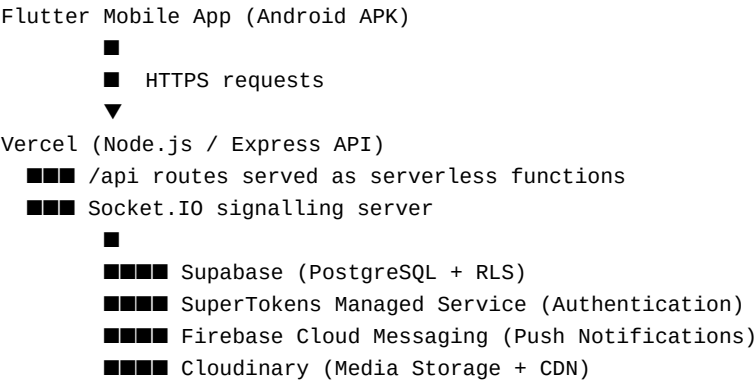


# Deployment Documentation

Resilio — Mobile Mental Health Platform for Nepal | Milan Shrestha | ID: 23057135 | 04\_Transition/Deployment.md

Documentation covering the deployment of the Resilio backend API to Vercel, including configuration, environment variables, deployment workflow, and known constraints.

## ARCHITECTURE OVERVIEW — DEPLOYED SYSTEM



## BACKEND DEPLOYMENT — VERCEL

### Why Vercel

| Reason                             | Detail                                                                                                          |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Zero-config Node.js deployment     | Vercel detects Express/Node.js projects and deploys without manual server configuration                         |
| Free tier sufficient for FYP scope | Hobby plan covers the load expected during UAT and testing                                                      |
| Automatic HTTPS                    | TLS certificates provisioned automatically — satisfies the security requirement that all API traffic uses HTTPS |

| Reason                          | Detail                                                                                                               |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Git-based continuous deployment | Every push to `main` triggers an automatic redeploy — no manual deployment steps                                     |
| Serverless functions            | API routes are served as serverless functions; no server instance management required                                |
| Environment variable management | Vercel dashboard provides a secure store for all secrets (Supabase key, SuperTokens key, Firebase credentials, etc.) |

## Vercel Project Configuration

`vercel.json` — placed in the root of the Node.js backend project:

```
{
  "version": 2,
  "builds": [
    {
      "src": "src/index.js",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "src/index.js"
    }
  ]
}
```

### What this does:

- Tells Vercel the entry point is `src/index.js`
- Routes all incoming requests to the Express app
- Uses `@vercel/node` runtime for Node.js 20.x

## Vercel Environment Variables

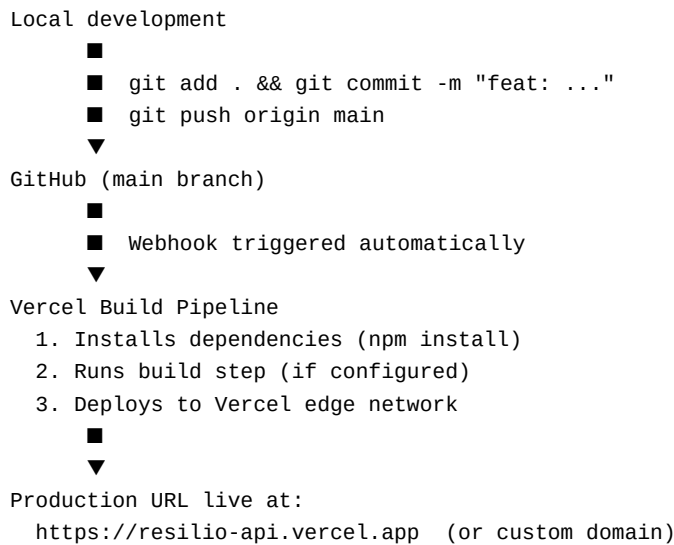
All secrets are stored in the Vercel project dashboard under **Settings** → **Environment Variables**, not in any committed file. The following variables are configured:

| Variable                | Description               |
|-------------------------|---------------------------|
| <code>`NODE_ENV`</code> | <code>`production`</code> |

| Variable                                  | Description                                                        |
|-------------------------------------------|--------------------------------------------------------------------|
| <code>`SUPABASE_URL`</code>               | Supabase project URL                                               |
| <code>`SUPABASE_SERVICE_KEY`</code>       | Supabase service role key (full database access, server-side only) |
| <code>`SUPERTOKENS_CONNECTION_URI`</code> | SuperTokens managed service URI                                    |
| <code>`SUPERTOKENS_API_KEY`</code>        | SuperTokens API key                                                |
| <code>`FIREBASE_SERVICE_ACCOUNT`</code>   | Firebase service account JSON (stringified)                        |
| <code>`CLOUDINARY_CLOUD_NAME`</code>      | Cloudinary cloud name                                              |
| <code>`CLOUDINARY_API_KEY`</code>         | Cloudinary API key                                                 |
| <code>`CLOUDINARY_API_SECRET`</code>      | Cloudinary API secret                                              |
| <code>`ESEWA_SECRET_KEY`</code>           | eSewa HMAC secret for payment verification                         |
| <code>`ESEWA_CLIENT_ID`</code>            | eSewa client ID                                                    |
| <code>`TURN_SERVER_URL`</code>            | TURN server address for WebRTC                                     |
| <code>`TURN_USERNAME`</code>              | TURN server credential                                             |
| <code>`TURN_PASSWORD`</code>              | TURN server credential                                             |

**Security note:** `SUPABASE_SERVICE_KEY` grants full database access bypassing RLS. It is only ever used server-side (Node.js) and is never exposed to the Flutter client. The Flutter client uses the `SUPABASE_ANON_KEY` which respects RLS policies.

## Deployment Workflow



**Deployment time:** Approximately 30–60 seconds from git push to live.

**Preview deployments:** Every pull request branch automatically gets a preview deployment URL (e.g. `https://resilio-api-git-feature-branch.vercel.app`) — used for testing features before merging to main.

---

## Socket.IO on Vercel — Constraint and Solution

**Problem:** Vercel's serverless function model terminates connections after each request. Socket.IO requires a persistent connection (WebSocket), which is incompatible with Vercel's standard serverless deployment.

### Solution options considered:

| Option                                                                  | Decision                                                                                |
|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Deploy Socket.IO on a separate persistent server (Railway, Render, VPS) | <b>**Selected**</b> — Socket.IO signalling server deployed separately from the REST API |
| Use Vercel's Edge Functions with WebSocket support                      | Not yet stable at time of development                                                   |
| Use Ably or Pusher as managed Socket.IO alternative                     | Added cost and vendor dependency                                                        |

**Implementation:** The Socket.IO signalling server runs as a standalone Node.js process on a separate cloud host (persistent VM or platform like Railway/Render). The REST API runs on Vercel. The Flutter client connects to both independently:

```
// REST API calls → Vercel
final _dio = Dio(BaseOptions(baseUrl: 'https://resilio-api.vercel.app'));

// WebRTC signalling → Socket.IO server (separate host)
final _socket = io('https://resilio-signalling.up.railway.app', ...);
```

---

## Flutter Client API URL Configuration

The production Vercel URL is configured in the Flutter app via a constants file:

```
// lib/core/constants/api_endpoints.dart
class ApiEndpoints {
  static const String baseUrl = 'https://resilio-api.vercel.app';

  // SuperTokens auth
  static const String stCreateCode = '$baseUrl/auth/signup/code';
  static const String stConsumeCode = '$baseUrl/auth/signup/code/consume';
  static const String passwordlessComplete = '$baseUrl/auth/passwordless/complete';

  // API routes
  static const String appointments = '$baseUrl/appointments';
  static const String mood = '$baseUrl/mood';
  // ... etc
}
```

During development, `baseUrl` is switched to `http://10.0.2.2:3000` (Android emulator localhost) or `http://[LAN_IP]:3000` (physical device testing).

---

## CORS Configuration

The Express app configures CORS to allow requests from the Flutter mobile client:

```
const cors = require('cors');

app.use(cors({
  origin: '*', // Mobile apps do not have a browser origin
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization', 'rid', 'anti-csrf'],
}));
```

**Note:** The `rid` and `anti-csrf` headers are required by SuperTokens' FDI protocol and must be included in `allowedHeaders`.

---

## Vercel Deployment Limitations Encountered

| Limitation                              | Impact                                                               | Resolution                                                                  |
|-----------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Serverless function cold starts         | First request after idle period takes 1–3 seconds longer             | Acceptable for academic project; production would use always-on instances   |
| 10-second function timeout (Hobby plan) | Long-running operations (e.g. large media processing) could time out | Media upload is handled directly by Flutter → Cloudinary (bypasses backend) |
| No persistent WebSocket (Socket.IO)     | Socket.IO signalling cannot run on Vercel                            | Separate host for signalling server                                         |
| 100 MB function size limit              | Bundling large packages could hit limit                              | Managed dependencies kept lean; no issue encountered                        |

## DEPLOYMENT CHECKLIST (Pre-Submission)

| Item                                               | Status                                                  |
|----------------------------------------------------|---------------------------------------------------------|
| All environment variables set in Vercel dashboard  | Done                                                    |
| Production Supabase URL and service key configured | Done                                                    |
| SuperTokens production connection URI active       | Done                                                    |
| Firebase service account JSON uploaded as env var  | Done                                                    |
| Cloudinary production credentials set              | Done                                                    |
| eSewa production credentials set                   | Done                                                    |
| CORS configured for mobile client                  | Done                                                    |
| HTTPS active on Vercel deployment URL              | Done (automatic)                                        |
| Socket.IO signalling server deployed separately    | Done                                                    |
| Flutter `baseUrl` updated to Vercel production URL | Done                                                    |
| End-to-end smoke test on production deployment     | Done — TC-S001 to TC-S004 re-run against production URL |

## RELATED DOCUMENTS

- `07_API_Reference.md` — all REST endpoints deployed on this Vercel instance
- `12_WebRTC_Signalling_Architecture.md` — Socket.IO server deployed separately from Vercel
- `09_Development_Tools_and_Environment.md` — local development environment setup